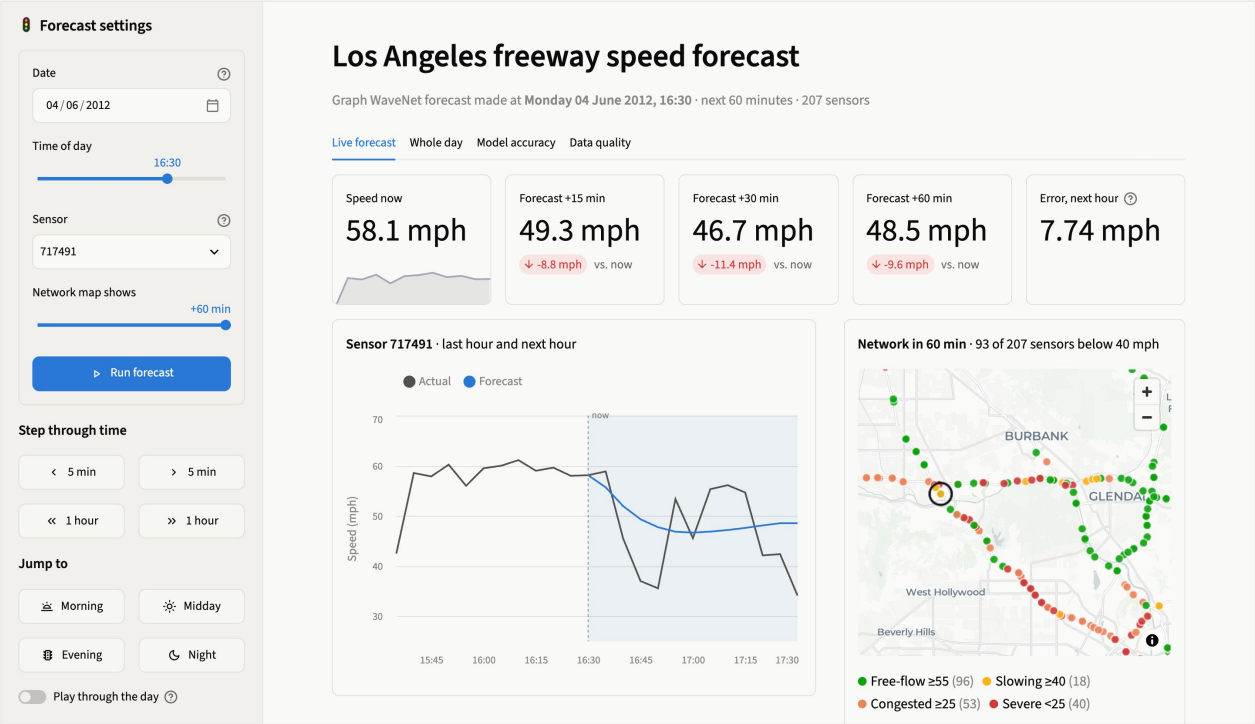
APPLIED MACHINE LEARNING • PROJECT REPORT

Predicting Los Angeles freeway traffic, one hour ahead

A machine-learning system that reads live speeds from 207 road sensors and forecasts how fast traffic will be moving at each sensor over the next 60 minutes. The forecasts can be explored in an interactive website.



Dataset
METR-LA · 207 sensors · Mar–Jun 2012

Best model
Graph WaveNet (graph neural network)

Website
Streamlit app · `app/app.py`

01 · AT A GLANCE

What this project does, in one page

Navigation apps, traffic control rooms and emergency services all need to know what traffic will look like *soon*, not just right now. This project builds and compares several machine-learning models that look at the last hour of traffic on Los Angeles freeways and predict the speed at every sensor for each of the next twelve 5-minute steps (5 to 60 minutes ahead).

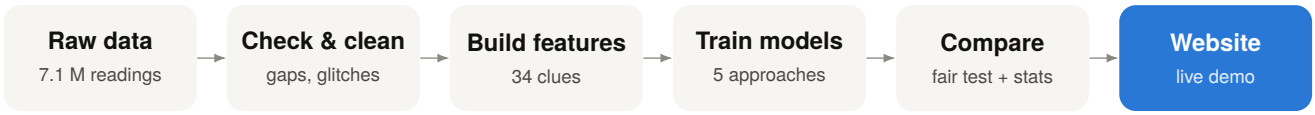
3.07 mph
Average forecast error of the best model, across all 12 steps

2.72 mph
Error 15 minutes ahead. Traffic is easier to predict in the short term

27%
Less error than the simple "usual speed for this time" guess

207
Sensors forecast at once, every 5 minutes

The journey, step by step



The question

Given the speed at all 207 sensors over the **last 60 minutes**, and a map of how the sensors are connected by road, what will the speed be at each sensor **5, 10, … 60 minutes from now**?

Why it is hard

- **Sudden jams.** Speed can drop from 65 to 20 mph within minutes.
- **Rush hours.** Traffic follows daily and weekly rhythms.
- **Jams spread.** A slowdown at one point moves backwards up the road to other sensors.
- **Broken sensors.** About 8% of readings are missing.

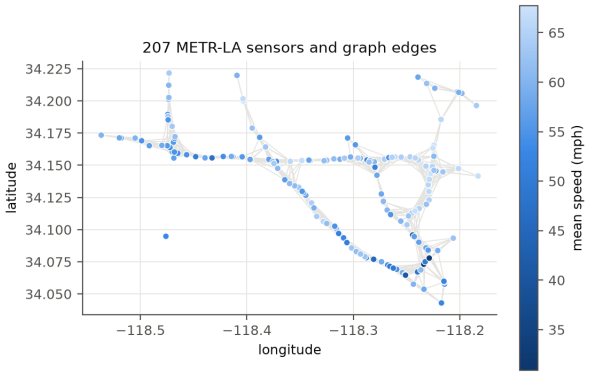
What we found

- A **graph neural network** (Graph WaveNet), which learns how sensors influence each other, was the most accurate model at every horizon.
- A much simpler **gradient boosting** model with well-designed inputs came close: only 0.07 mph behind at 60 minutes.
- Statistical tests confirm the best model's lead is real, not luck ($p < 0.001$).
- Our result is within 0.03–0.06 mph of the published Graph WaveNet paper, so the implementation is faithful.
- All of it can be explored in a **web app** that replays any moment of the test period.

02 · THE DATA

207 sensors, four months, every five minutes

We use **METR-LA**, the standard public benchmark for traffic forecasting. Loop detectors buried in the road on Los Angeles County freeways record the average vehicle speed every 5 minutes. Because so many research papers use it, our numbers can be compared directly with published work.

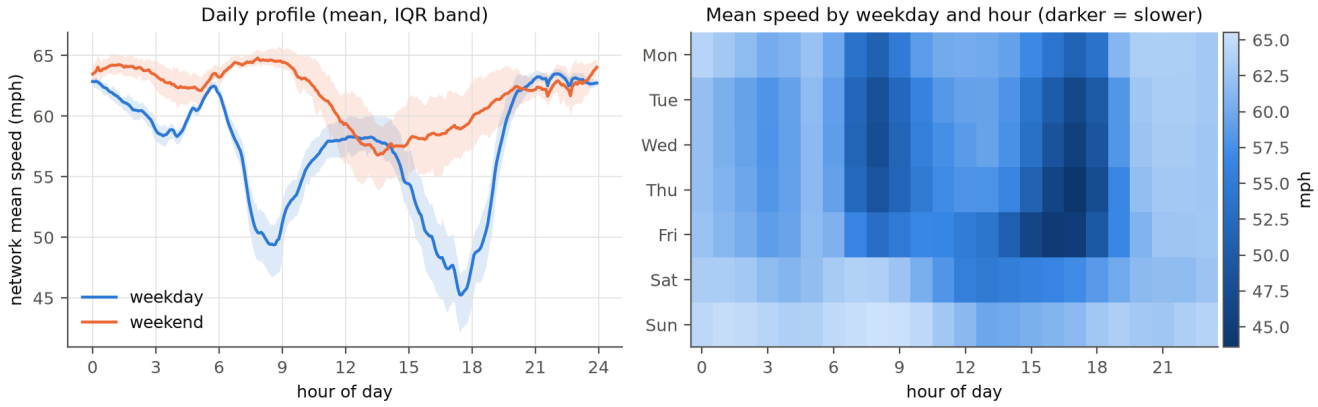


Where the sensors are. Each dot is a sensor, coloured by its average speed (darker is slower). The grey lines join sensors that are close along the road. These links form the "graph" the best model uses.

PROPERTY	VALUE
Sensors	207
Period	1 Mar – 27 Jun 2012
Readings per sensor	34,272 (119 days)
Total readings	7.09 million
Average speed	58.5 mph
Speed limit of the sensors	capped at 70 mph
Readings under 40 mph (jams)	10.9%
Missing readings	8.1%
Road links between sensors	1,515

The data also comes with the GPS location of each sensor and the driving distance between pairs of sensors. We use those distances to build the road map.

What exploring the data told us



The rhythm of the week. Left: average network speed across the day. Weekdays (blue) have a clear morning dip around 8:30 and a deeper evening dip around 17:30; weekends (orange) stay fast. Right: the same pattern by day of the week. Darker cells are slower, so Tuesday–Friday evenings are the worst.

FINDING

Strong weekly rhythm

Each sensor's "usual speed for this time of the week" is a powerful clue, so we turned it into a feature.

FINDING

Recent past matters most

Speed 5 minutes ago is 86% correlated with speed now, but that link fades fast. Long-range forecasts need more than the last reading.

FINDING

Neighbours move together

Sensors joined by road are about 4× more correlated in their unusual swings than unconnected ones. This is why a road-map-aware model helps.

03 · DATA QUALITY

Checking and cleaning the readings

Real sensors fail. Before modelling, every reading went through a set of automatic checks. Missing values show up in the file as **0 mph**, which is easy to mistake for a total standstill, so they have to be found and handled carefully.

What was checked

- **12 integrity checks**: timestamps exactly 5 minutes apart, no duplicate sensors, every speed between 0 and 70 mph, every sensor inside Los Angeles, and so on. All 12 passed.
- **Network outages**: moments when *all* 207 sensors read 0 at once (2,148 time steps, 6.3% of data).
- **Single sensor missing**: one sensor reads 0 while its neighbours are fine (1.8%).
- **Spikes**: a one-off jump far from the surrounding readings (6,142 readings).
- **Stuck sensors**: the same value repeated for over an hour (274 readings).

How gaps were filled

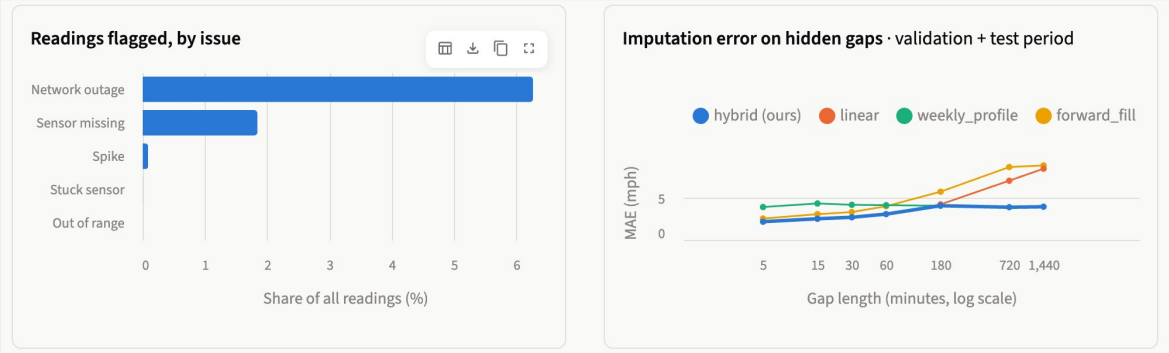
We hid random stretches of good data, filled them in with several methods, and measured which guessed best:

- **Short gaps (up to 1 hour):** draw a straight line between the readings on either side.
- **Long gaps:** use the sensor's usual speed for that time of the week. For a 24-hour gap this was twice as accurate (4.0 vs 8.6 mph error).

Important detail. Good readings are never changed. And the models are still **scored only on real readings**, never on filled-in ones, so that our results stay comparable with published research.



Website, "Data quality" tab. One sensor on one day: raw readings (grey), the cleaned series (blue) and flagged readings (orange crosses).



Lower half of the same tab. Left: share of readings flagged, by type. Network outages dominate. Right: error of each gap-filling method as gaps get longer. Our hybrid rule (blue) stays lowest across all gap lengths.

04 · THE MODELS

Five ways to forecast, from simple to smart

To know whether a complex model is worth it, you need simple ones to beat. We built a “ladder” of five forecasters. Every model sees exactly the same data and is scored the same way.

BASELINE

1 · Historical Average

“Traffic will be at its usual speed for this time on this weekday.” Ignores what is happening right now.

BASELINE

2 · Persistence

“Speed will stay exactly as it is now.” Hard to beat 5 minutes ahead, poor an hour ahead.

CLASSICAL ML

3 · Ridge regression

A weighted sum of 34 hand-made clues (features). Fast and easy to explain, but can only learn straight-line relationships.

CLASSICAL ML

4 · Gradient Boosting

Hundreds of small decision trees, each fixing the previous ones’ mistakes. Uses the same 34 clues but can learn curved, “if-this-then-that” patterns.

DEEP LEARNING · BEST MODEL

5 · Graph WaveNet

A graph neural network. It treats the sensors as points on a network and learns how traffic at one sensor affects others. It uses the real road map, plus a second, **self-learned map** that can find links the roads alone don’t show. Over time, it uses stacked convolution layers that look back at different time scales. It has about 301,000 learnable numbers and took 2.4 hours to train on a laptop (Apple M3).

The 34 clues given to the classical models

Deep models discover patterns by themselves; classical models need them spelled out. We built six groups of features. Every one uses **only past information**, and automated tests check that no future data leaks in.

GROUP	PLAIN-LANGUAGE MEANING
Lags	Speed now, 5 min ago, 10 min ago … up to an hour ago
Window stats	Average, spread and minimum over the last hour; how many readings were missing
Calendar	Time of day and day of week of the moment being predicted
Weekly profile	This sensor’s usual speed at this time of week, and how far today differs from usual
Spatial	Average speed of the sensors just upstream and just downstream on the road
Sensor	Fixed facts about the sensor, such as its overall average and how connected it is

How much each group helped

Gradient Boosting, 60 min ahead. Groups were added one at a time; bars show the error (mph) after each addition. Shorter is better.



Knowing **“what is normal for this sensor at this time”** was the single most valuable clue. Altogether the features cut the 60-minute error by 22%.

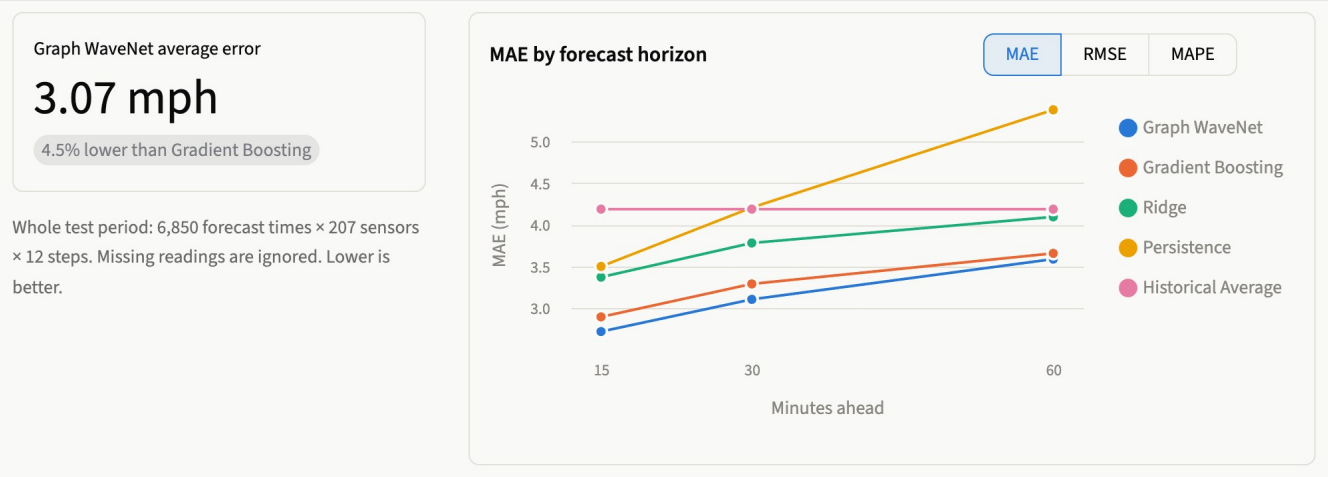
05 · RESULTS

Which model forecasts best?

The models were trained on the first 70% of the timeline (March to May), tuned on the next 10%, and tested on the **last 20%** (4–27 June), which they had never seen. Splitting by time like this mimics real use: you always predict the future from the past.

MODEL	15 MIN	30 MIN	60 MIN	AVERAGE
Graph WaveNet	**2.72**	**3.10**	**3.59**	**3.07**
Gradient Boosting	2.89	3.29	3.66	3.22
Ridge regression	3.37	3.78	4.09	3.69
Persistence	3.50	4.21	5.38	4.24
Historical Average	4.19	4.19	4.19	4.19

Mean absolute error (MAE) in mph: on average, how far the forecast was from the real speed. Lower is better. Missing readings are ignored.



Website, "Model accuracy" tab. Error rises the further ahead we predict. Persistence (yellow) gets worse fastest, Historical Average (pink) stays flat, and Graph WaveNet (blue) is lowest at every horizon. The chart can be switched between MAE, RMSE and MAPE, and the tab also lists the full table above.

Is the win real?

Small differences can be luck. We used the **Diebold–Mariano test**, a standard way to compare two forecasters, and **bootstrap confidence intervals**, which resample whole days to see how stable each score is. Graph WaveNet beats every other model at every horizon with $p < 0.001$.

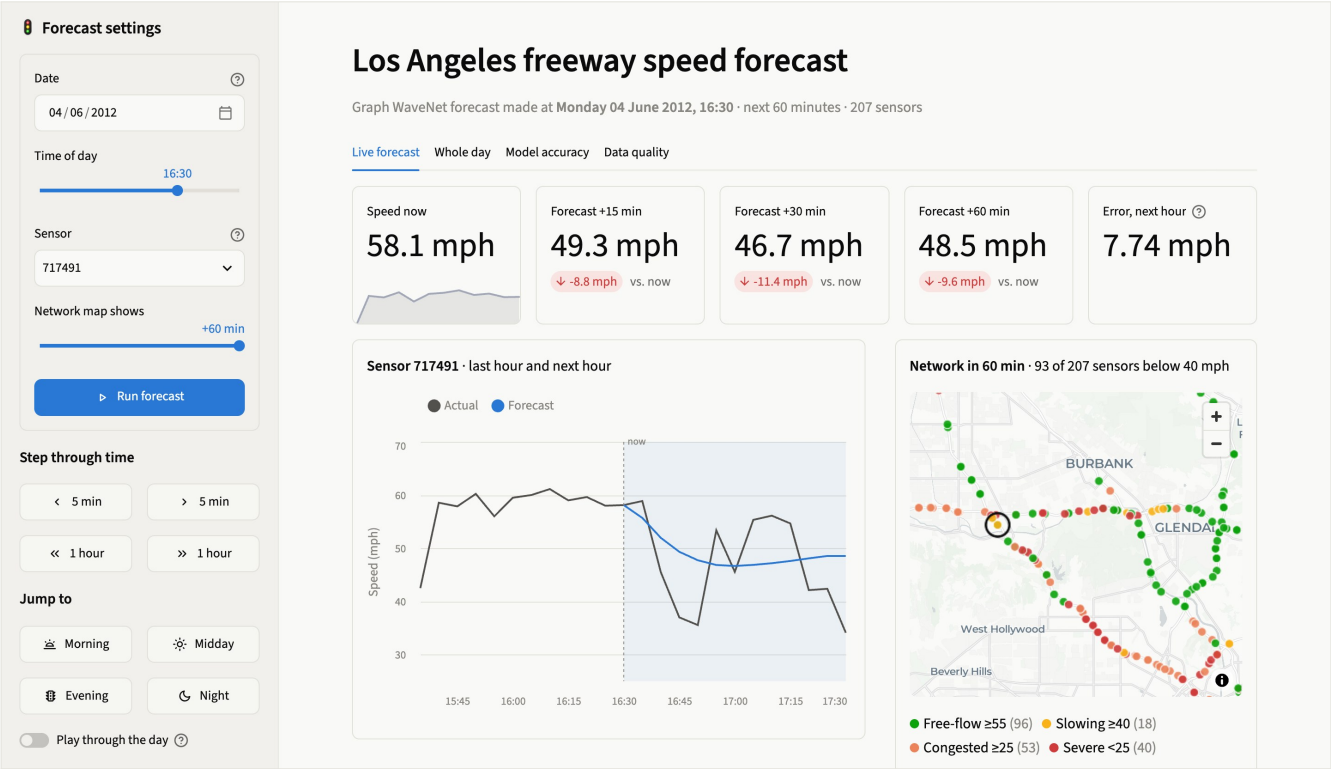
How does it compare to research?

The original Graph WaveNet paper reports 3.53 mph at 60 minutes; ours reaches 3.59. The well-known DCRNN model reports 3.60, so **our model matches published state-of-the-art results** under the same test protocol.

06 · THE WEBSITE

An interactive forecast dashboard

The trained Graph WaveNet model powers a web app built with **Streamlit**. You pick any moment in the test period, and the app runs the model on the hour of data before it, then shows the forecast next to what really happened.



"Live forecast" tab, Monday 4 June 2012, 16:30. The top cards show the speed now and the forecasts for +15, +30 and +60 minutes at the chosen sensor. The chart compares the forecast (blue) with what was really measured (grey). The map colours every sensor by its predicted speed in 60 minutes. Here 93 of 207 sensors are expected to drop below 40 mph as the evening rush builds.

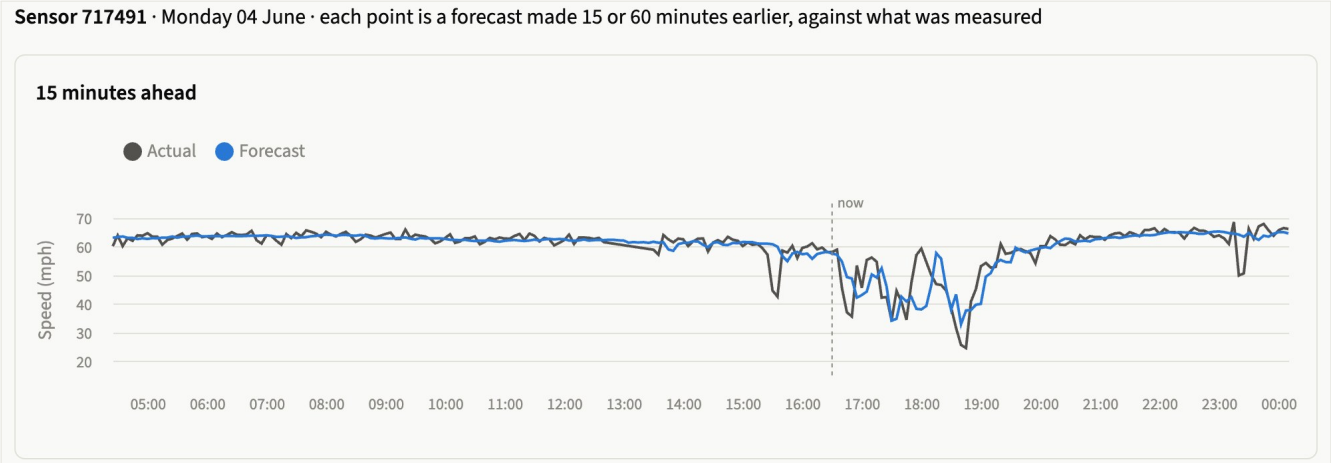
Controls (left sidebar)

- **Date, time and sensor** pickers, then *Run forecast*.
- **Step through time** by 5 minutes or an hour, or **jump** to morning, midday, evening or night.
- **Play through the day** moves the clock forward automatically so you can watch jams form.
- **Find a sensor**: the slowest right now, the one about to slow down most, the one with the biggest error, or a random one. You can also click any dot on the map.

Four tabs

- **Live forecast**: the next hour at one sensor plus the network map.
- **Whole day**: every forecast for the day, made 15 or 60 minutes ahead.
- **Model accuracy**: all five models compared on the full test set.
- **Data quality**: cleaning results, flagged readings and the least reliable sensors.

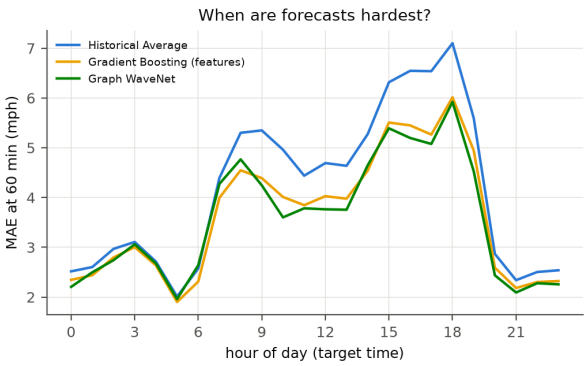
Map colours: free-flow ≥ 55 · slowing ≥ 40 · congested ≥ 25 · severe < 25 mph.



"Whole day" tab. Every forecast made 15 minutes ahead for a full day at one sensor (blue) against what was really measured (grey). The model follows the evening slowdown closely. A second chart below it (not shown) does the same for 60 minutes ahead, where the model still predicts the dip but smooths out the sharpest drops.

07 · LESSONS AND NEXT STEPS

Where the model struggles, and what we learned



Error by time of day (60 min ahead). Night is easy; the evening rush around 18:00 is hardest for every model.

- **Rush hour is hardest.** Errors peak near 6 mph around 18:00, about three times the night-time level, because jams start and clear suddenly.
- **Missing inputs hurt the deep model more.** When a sensor's latest reading is missing, Graph WaveNet's 60-minute error rises to 5.36 mph, while Gradient Boosting rises only to 4.50. The hand-made "missing" features help.
- **Simple can be strong.** Gradient Boosting trains in about 10 minutes on a CPU and is nearly as good an hour ahead.
- **Next steps:** add weather, incidents and events; train with simulated sensor failures; try newer Transformer-based models; serve live data instead of replaying 2012.

Conclusion

The project covers the full machine-learning pipeline: exploring and cleaning real sensor data, designing leak-free features, training five models of increasing complexity, testing the results statistically, and putting the best model in front of users. The graph neural network wins because it learns how traffic flows between connected sensors. A well-designed classical model comes surprisingly close.

Run it yourself

```
pip install -r requirements.txt && pip install -e .
pytest                                                  # 32 automated tests
streamlit run app/app.py                                # open the website
```

Code: Python package in `src/traffic/`, one script per experiment in `scripts/`, all results in `results/`. Key references: Li et al., *DCRNN*, ICLR 2018; Wu et al., *Graph WaveNet*, IJCAI 2019.